

Reverie — Presentation Script

Speaker script for `reverie-deck.html` (25 slides). Split across the five team members per the agreed block assignment. Target runtime about 15 minutes. Every number below already appears on a slide and traces back to `results/log.csv`, `EXPERIMENTS.md`, and the notebooks.

Presentation date: July 2, 2026 **Team:** Amaly Attia · Em Echeverria · Lea Sarouphim Hochar · Stephan Pentchev · Cecile Tambey

Who presents what

Slide numbers below are the actual deck positions, 1 to 25, including the three section dividers.

Slides	Presenter	Section
1 to 5	Em Echeverria	Opening and business case
6 to 8	Amaly Attia	System and data
9 to 13	Stephan Pentchev	The two models
14 to 20	Lea Sarouphim Hochar	Evidence
21 to 25	Cecile Tambey	MVP, reflection, and close

Approximate timing: Em about 3 min, Amaly about 2 min, Stephan about 3.5 min, Lea about 3.5 min, Cecile about 3 min.

Two credits to land out loud: Stephan ran the sweeps and ablation behind slides 16 and 17; Lea presents them as part of the evidence story, so she names Stephan there. Em built the collaborative model and the frontend; Stephan names her on slide 11, and Cecile names her on the demo, slide 23, where Em fields any questions.

Em Echeverria — Opening and business case (slides 1 to 5)

Slide 1 — Title

“Good afternoon. We are Reverie, and our project answers one very ordinary question: what should I watch next? I am Em, and I will set up the problem and the product. My teammates will take you through the data, the two models, the evidence, and the live app. Let me start with why this is worth solving.”

Slide 2 — The problem

“Streaming gave us tens of thousands of titles and somehow made watching harder. The average viewer now spends roughly ten minutes per session just deciding, and a lot of people give up without watching anything. For a platform that decision fatigue is not a small annoyance. It shows up as churn. So the pain is real on both sides of the screen.”

Slide 3 — The value proposition

“Reverie replaces ten minutes of scrolling with a ranked shortlist in under a second. You import what you have already seen, and the model ranks the entire catalog to your taste. And because choosing a film with someone else is the harder problem, the Blend does it for two people at once.”

Slide 4 — Precision

“Here is the number we are proudest of. On the high confidence picks, the ones the model labels you will love it, precision was one point zero across held out real viewers. Plainly: when Reverie was sure, it was right. A recommender earns trust by being honest about its best calls, and this is that.”

Slide 5 — Business impact

“To frame the money, and these are illustrative industry figures, not a claim about our model: Netflix has publicly attributed about one billion dollars a year to its recommender through reduced churn. Scale that down. A single point of churn cut on one million subscribers at fifteen euros a month is about one point eight million euros a year retained. Personalization is the lever, and Reverie is that lever productized. Amaly will now show you the system underneath it.”

Amaly Attia — System and data (slides 6 to 8)

Slide 6 — Divider: How we built it

“Thanks Em. I am Amaly, and I own how data gets into Reverie and how the pieces connect. Let me walk the system end to end.”

Slide 7 — System architecture

“Read this left to right. On the left are our sources. MovieLens gives us watch sequences. Two open Letterboxd datasets give us nine point nine million ratings and the poster and metadata layer. TMDB fills in genres and ratings. And the last source is you: I built the importers that read a Letterboxd or a Netflix export, resolve every title to our catalog, and fold in a TV catalog so we cover shows and not just films. That title resolution and data verification step is what keeps the rest of the pipeline honest.

Moving right, the data layer turns all of that into artifacts: trained weights, embeddings, and a catalog file, plus a SQLite database for accounts, history, watchlists, and friends. In the center sit the two deep learning models. To their right is the FastAPI service, and on the far right the React app. The simple way to read it: everything left of the models is data, everything right of them is the product.”

Slide 8 — Two models

“And there are exactly two networks, because the product asks two different questions. Model one is a GRU, a recurrent network. It asks what comes next in your sequence, and it answers with a softmax over the catalog. Model two is a collaborative network with embeddings. It asks how much you will like a given film, and it answers with a predicted rating from one to ten. One is about order, the other is about taste shared across everyone. Stephan will take you inside both.”

Stephan Pentchev — The two models (slides 9 to 13)

Slide 9 — Model 1, the recurrent network

“Thanks Amaly. I am Stephan and I built and tuned the models. Start with the GRU. The key idea is a cell that remembers. We feed it your last twenty films as a sliding window. Each film becomes an embedding of thirty two dimensions, combined with its genre and rating channels. The GRU cell, sixty four units with its reset and update gates, folds each step into a hidden state that it passes forward. After dropout, the head is a softmax over the whole catalog. So this is a classification problem trained with cross entropy, and the three concepts from class are right here: the embeddings, the memory loop, and the softmax.”

Slide 10 — Model 1 unrolled

“Unrolled through time it looks like this. Film one updates the state, which feeds film two, and so on, until the model predicts the next film. We trained on every prefix, so each prefix predicts its own next item, and we evaluated leave one out, holding out each user’s last film. All statistics are computed on training data only, so there is no leakage. The configuration we locked, chosen on validation, was history twenty, embedding thirty two, sixty four units, GRU over LSTM.”

Slide 11 — Model 2, the collaborative network

“The second model is feed forward. Em built this one as the product extension, and it works on embeddings. A user id becomes a fifty dimensional vector, a movie id becomes another fifty, and a frozen content tower of twenty three features carries genres and numerics. We concatenate them, push through a dense sixty four with ReLU, a dense thirty two with ReLU and dropout, and a final single unit that outputs a rating. Those dense ReLU layers are the hidden layers that learn what makes you you. The loss is mean squared error, reported as RMSE, the strict ruler.”

Slide 12 — Model 2 forward pass

“One design choice worth calling out. The model is bias aware. The prediction is the global mean, plus a user bias, plus a movie bias, plus the network’s output. The biases recover the easy baseline on their own, which frees the embeddings to learn only the residual, the part that is actually personal. That is why it trains cleanly and beats the naive means.”

Slide 13 — Serving

“Now the honest part. In the live app we do not serve a cold fold in of a brand new user vector, because we tested it and it underperformed. Instead the app ranks each film by its similarity to your highest rated films in the learned embedding space, plus a genre anchor and a small recency nudge. The reason is simple: the embedding space is well trained, but a fresh user vector built from a few ratings is not. We reached that by testing, not by assuming. Lea will show you whether all of this actually works.”

Lea Sarouphim Hochar — Evidence (slides 14 to 20)

Slide 14 — Divider: Does it work?

“Thanks Stephan. I am Lea, and I ran the official test and the baselines. So let me answer the only question that matters: does it work?”

Slide 15 — Overfitting prevention

“First, we did not just hope it generalized, we watched it. This is our real GRU run. Training loss keeps falling, but validation loss bottoms out at epoch sixteen and then starts to climb. That gap is overfitting, live. Early stopping catches it and restores the best weights from epoch sixteen. So nothing here is the model memorizing the training set.”

Slide 16 — Hyperparameter tuning

“Next, why the settings are what they are. These choices came out of the sweeps Stephan ran. On history length, twenty wins on HR at ten, ahead of ten and fifty. We then kept the smaller network, embedding thirty two and sixty four units, because it had a tighter train to validation gap, and we kept the GRU over the LSTM because it scored the same with fewer parameters.”

Slide 17 — Feature ablation

“We also checked that the inputs earn their place. This is the E6 ablation, again from Stephan’s runs. Adding genre helps. Rating on its own actually hurts ranking. But the full hybrid is the version we ship, because it wins on MRR at zero point one three five, and the rating channel is exactly what lets a thumbs down re-rank your list live. So we trade a hair of HR for a feature the product needs.”

Slide 18 — GRU baselines

“Now the headline ranking result. Our GRU reaches HR at ten of zero point two five seven, against zero point zero seven six for item k nearest neighbors and zero point zero four two for most popular. On the frozen test across three seeds we get zero point two four two plus or minus zero point zero zero two, with non overlapping ninety five percent confidence intervals against item k nearest neighbors, which is the strong baseline that shallow recurrent models often lose to. We beat it cleanly.”

Slide 19 — NCF baselines

“For the rating model, lower RMSE is better. Global mean sits at two point zero seven, user mean at one point nine six, movie mean at one point six two, and our neural net at one point three seven. It beats the movie mean baseline for ninety four percent of users, with a median per user RMSE of one point three two. Cold start is still hard, and we say so rather than hide it.”

Slide 20 — Confusion matrix

“And this is the precision claim made concrete. On a held out viewer’s most recent films, every single title the model flagged as a love actually was one. Zero false alarms, so precision is one point zero. It is conservative, it misses some, but the short list it does hand you is trustworthy, and that is the property a recommender actually needs. Cecile will now show you the product this drives.”

Cecile Tambey — MVP, reflection, and close (slides 21 to 25)

Slide 21 — Divider: The MVP

“Thanks Lea. I am Cecile, and I pulled the whole project together into what you are about to see. Let me take you through the working product and close us out.”

Slide 22 — Integration

“This is how it all connects in real time. The React app sends your history to a recommend endpoint, the model scores it, and ranked picks come back in under a second. The piece to notice is the feedback loop: a thumbs down appends to your live history as a low rating, the model re-runs, and the list re-ranks instantly, with no retraining at all.”

Slide 23 — Live demo

“Here it is in action. (Play `demo.mov`.) You import a history, the deck tunes to each pick you rate, and watch what happens when a film gets rejected: the next set shifts in real time. That is the model arguing back, not a static list. Em built the frontend and the API behind this, so if you want the technical details, she will take those in questions.”

Slide 24 — Reflection

“Three honest lessons. First, collaborative signal beat content: a single taste profile overfits, but learning from everyone generalizes. Second, genre tags are weak on their own, drama alone covers half of all films, which is why we rank by nearest favorites instead. Third, honest baselines mattered more than one flashy number.

We are also clear about the known issues. Cold start leaves a brand new viewer sitting near their own average. The embeddings lean toward a Letterboxd cinephile cluster. And our posters hotlink the Letterboxd CDN, which is a fragile dependency.

Where we go next: per user fine tuning as a history grows, a dial between mainstream and deep cuts, and a group Blend for more than two people.”

Slide 25 — Thank you

“That is Reverie: two neural models, a real working app, and results we can stand behind. Built on open data from MovieLens, the two Letterboxd datasets, and TMDB. Thank you, and we are happy to take questions.”

Demo backup note: the demo is recorded as `demo.mov` . If it does not play, export to mp4 H.264 and update the source.

Q&A backstop, by owner

- **Data, importers, TV titles** to Amaly.
- **GRU architecture, tuning, serving choice** to Stephan.
- **Baselines, test protocol, metrics** to Lea.
- **Frontend, API, the live re-rank, the collaborative model** to Em.
- **Scope, timeline, business framing** to Em or Cecile.